facecam

A lightweight, local face detection and recognition tool · White paper · October 2026

1. Summary

facecam is a small Python program that detects faces in a webcam stream or photo and decides whether each face belongs to an enrolled person, such as its owner. It runs entirely on the user's own computer: no cloud service, account or network connection is needed after two small model files are downloaded once. It is built on OpenCV and two compact pretrained neural networks, which keeps installation to a single `pip install` and lets it run on an ordinary laptop CPU.

2. Problem and goals

Face recognition tools are often heavy to install (compiled dependencies such as dlib), tied to paid cloud APIs, or opaque about where face data goes. facecam targets a different point in the design space:

- **Accurate** — use modern, purpose-built detection and recognition models rather than classical Haar cascades.
- **Private** — all processing and storage stay on the local machine.
- **Simple** — two dependencies (OpenCV, NumPy) and three commands.
- **Understandable** — roughly 250 lines of readable code split into three modules.

3. How it works

Each video frame passes through a three-stage pipeline:

Stage	Component	What it does
1. Detect	YuNet	Finds faces and returns a bounding box, five landmarks (eyes, nose, mouth corners) and a confidence score.
2. Embed	SFace	Uses the landmarks to align and crop the face, then converts it into a 128-number vector (an *embedding*) that captures identity rather than pose or lighting.
3. Match	Cosine similarity	Compares the vector against stored embeddings of enrolled people. A score at or above 0.363 (the threshold recommended by OpenCV for SFace) counts as a match; otherwise the face is labelled *Unknown*.

Enrollment. The user looks at the camera while the program captures 15 samples over a few seconds. A sample is accepted only when exactly one face is visible with detection confidence above 0.9, which prevents bystanders or poor detections from polluting the database. Multiple samples across slight head movement make matching more robust than a single photo. Only the embeddings are saved (`data/faces.npz`); no images are stored.

4. Usage and architecture

```
pip install -r requirements.txt
python -m facecam enroll --name Me   # capture 15 samples
python -m facecam run                # live view: green = match, red = Unknown
python -m facecam image photo.jpg    # annotate a single photo
```

Module	Responsibility
`facecam/cli.py`	Command-line commands (enroll, run, image) and on-screen drawing.
`facecam/models.py`	Model download and loading; detect, embed and identify functions; reading and writing the face database.
`facecam/config.py`	Paths, model URLs and tunable thresholds in one place.

5. Privacy and security considerations

- **Local only.** Video never leaves the machine. The only network use is the one-time model download from the OpenCV Zoo repository on GitHub.
- **Embeddings, not photos.** The database holds numeric vectors. These are still biometric data and should be treated as sensitive; the `data/` folder is excluded from version control for this reason.
- **Not a security lock.** A printed photo or a screen replay can fool a system without liveness detection. facecam must not be used as the sole control protecting anything valuable.
- **Consent.** Enroll only people who have agreed to it, and check local law before recognising other people's faces.

6. Limitations

- **No formal accuracy benchmark yet.** The models are published, pretrained components, but this project has not measured its own false-accept and false-reject rates. Results depend on camera quality, lighting and angle.
- **No liveness / anti-spoofing** (see above).
- **CPU-only, per-frame processing.** Speed depends on the machine and image resolution; there is no tracking between frames.
- **Threshold is global.** One cutoff applies to everyone; it should be tuned for the user's own environment.

7. Roadmap

- A small evaluation script that reports false-accept / false-reject rates on the user's own photos, so the threshold can be tuned with evidence.
- Optional liveness checks (blink or head-turn challenge) to resist photo attacks.
- Logging of recognition events with timestamps and optional snapshots of unknown faces, supporting the security-camera use case the project began with.
- Frame skipping and face tracking for lower CPU use on small devices.

8. Conclusion

facecam shows that accurate face recognition does not require a cloud service or a complex stack. By combining two compact pretrained models with a clean, small codebase, it gives individuals a private, inspectable tool for recognising their own face. Its main open items are measuring accuracy and adding spoofing resistance before it is relied on for anything security-critical.